MALWARE – PART II

Prof. Dr. Bernhard Tellenbach

Content

- Malware Defense
- Detection techniques
  - Signatures
  - Heuristics
  - Behavior
  - Reputation
- Evasion
- How good is Anti-Virus?

Goals

- You have a basic understanding of why our defenses against malware are still quite weak
- You can explain why anti-virus software is still an effective tool against (some) malware
- You know how anti-virus software works (signatures, fuzzy-hashes,…) and you can discuss its limitations

Zurich University
of Applied Sciences

zh
aw

Malware Defense

Quiz on Detecting Malware

- Can a tool detect 100% of all viruses that enter your system in form of a file?

- What's the problem with this approach?

Detecting Malware - Are we Good at it?

Why Aren't we Better at (Malware) Defence?

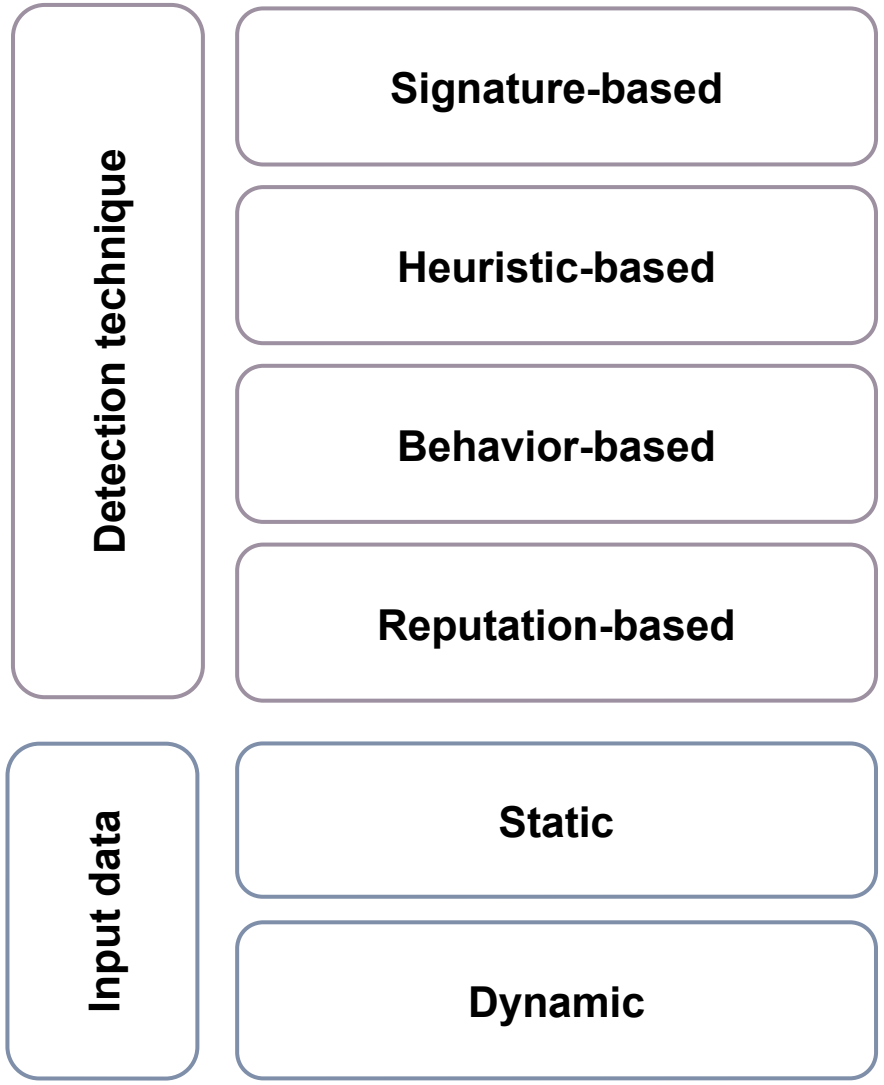
- For one single offensive LOC defenders write 100'000 LOC
  - 120:1 Stuxnet to average malware
  - 500:1 Simple text editor to average malware
  - **100,000:1** Defensive tool to average malware
  - 1,000,000:1 Target operating system to average malware

Positive aspect: Few LOCs make the analysis of (small) malware samples practical.

- Prospect theory "we are risk adverse when it comes to gains and risk taking when it comes to losses" (=>we don't act until it is too late!)
- Detecting malware collected in the wild is often «easy» as long as the characteristics used for the detection do not change.
- Chicken and egg problem - To know how to detect a malware, it must be analyzed. But to analyze it, we must find a sample of it.

Detecting Malware - Overview

- Anti-Malware software uses multiple detection engines
- Detection engines differ in the detection technique employed
- Detection engines differ in how they obtain the required input data
- One way to classify engines is by combining these two features
- Note: Hybrid forms are quite common, especially with regard to input data



Detecting Malware - Input Data Collection

- Static analysis: Data obtained without executing the sample
- Dynamic analysis: Data obtained while the sample is being executed
- The sample might be run natively, in an emulator, or a VM
- Data collection might be done at the same or at a different layer
  - E.g., run a sample in a VM and collect and analyze data from the outer layer (host)
- If at a different layer, malware has a harder time to detect being analyzed

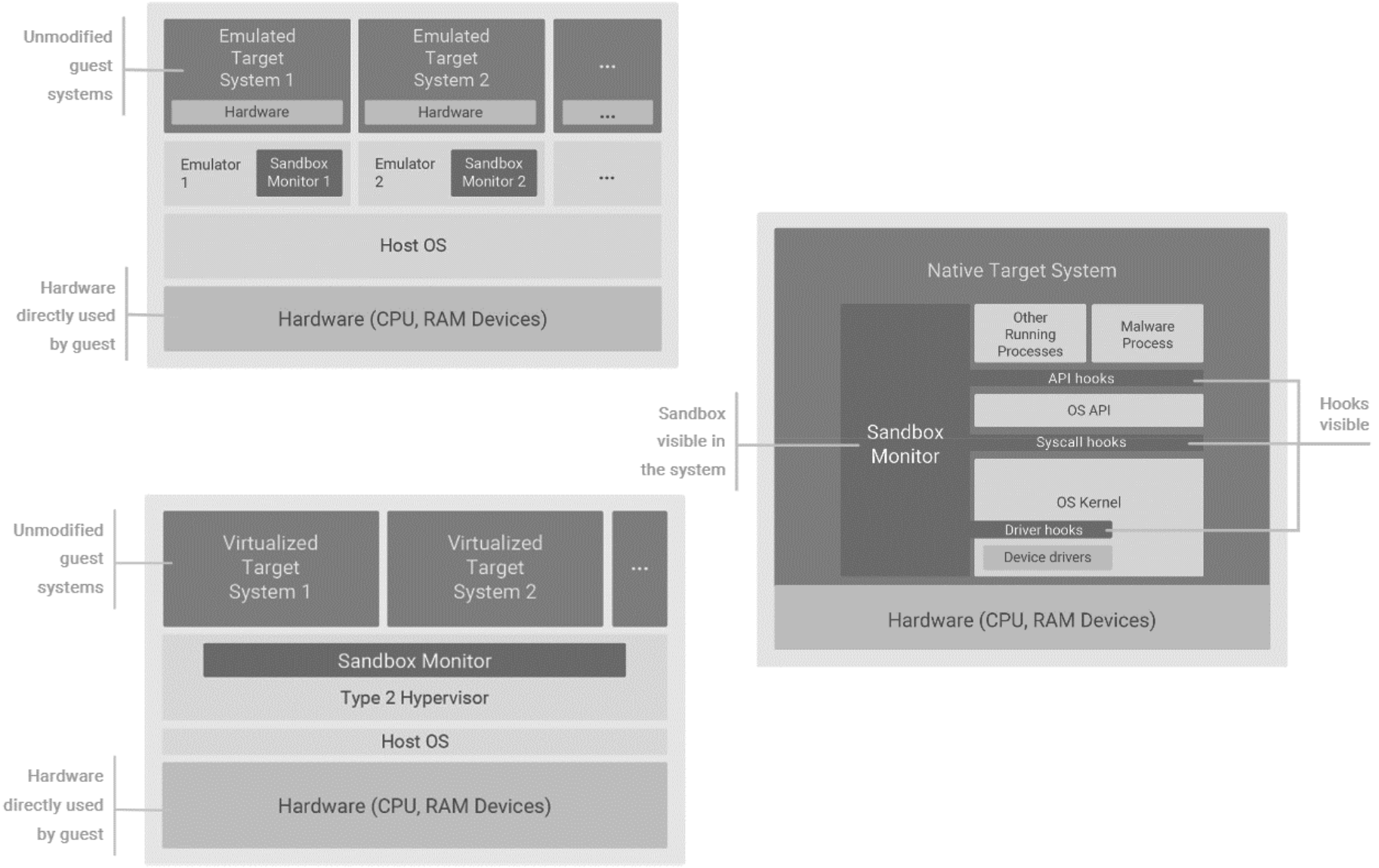
Static

- File metadata (e.g., PE header)
- Binary data/code
- Assembly code
- Assembly code characteristics
- API / System calls
- …

Dynamic

- Memory
- Network access/traffic
- Filesystem
- API / System calls
- …

Natively (with hooking), Emulation and Virtualization



Zurich University
of Applied Sciences

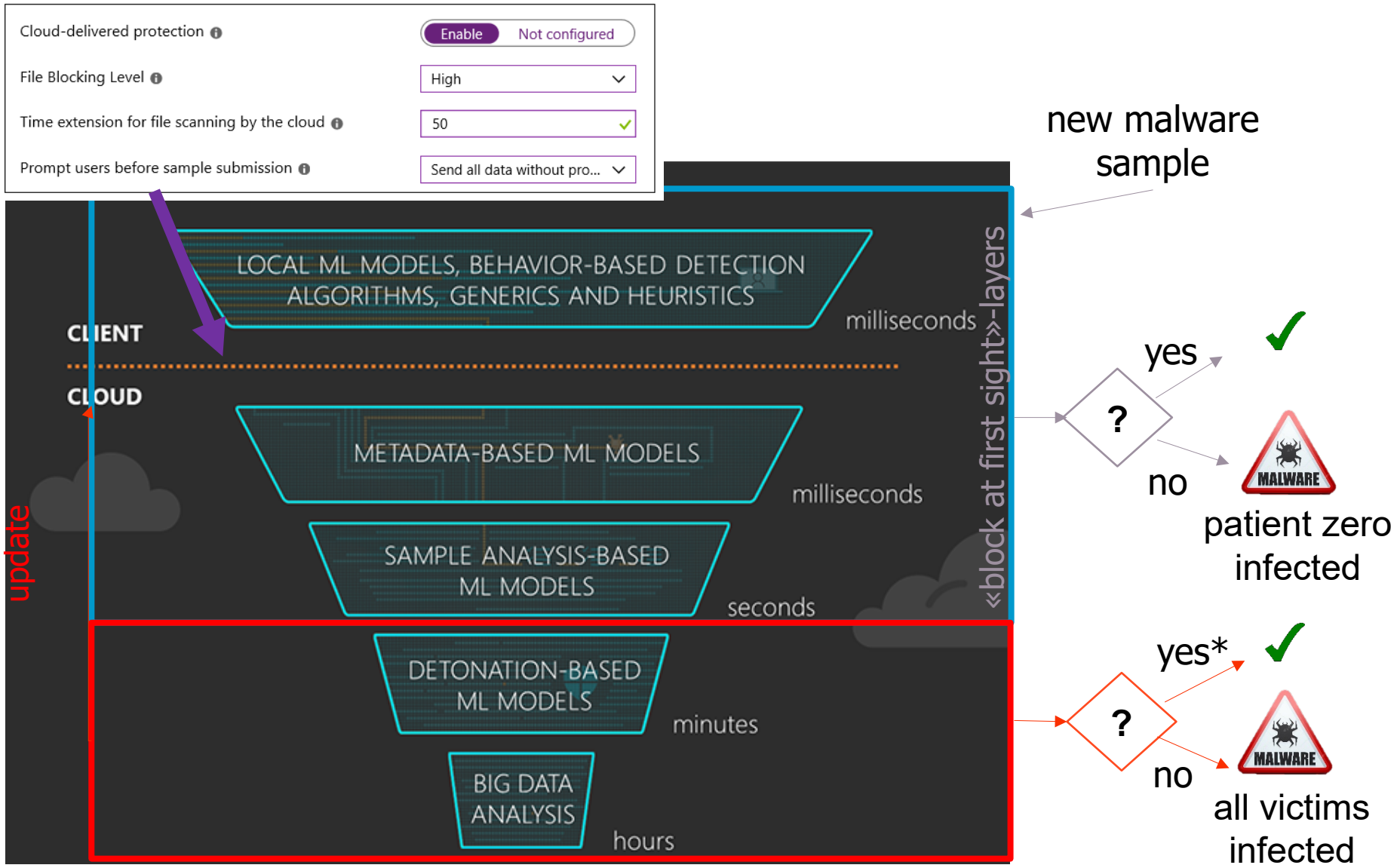
zh
aw

AV-System Architecture

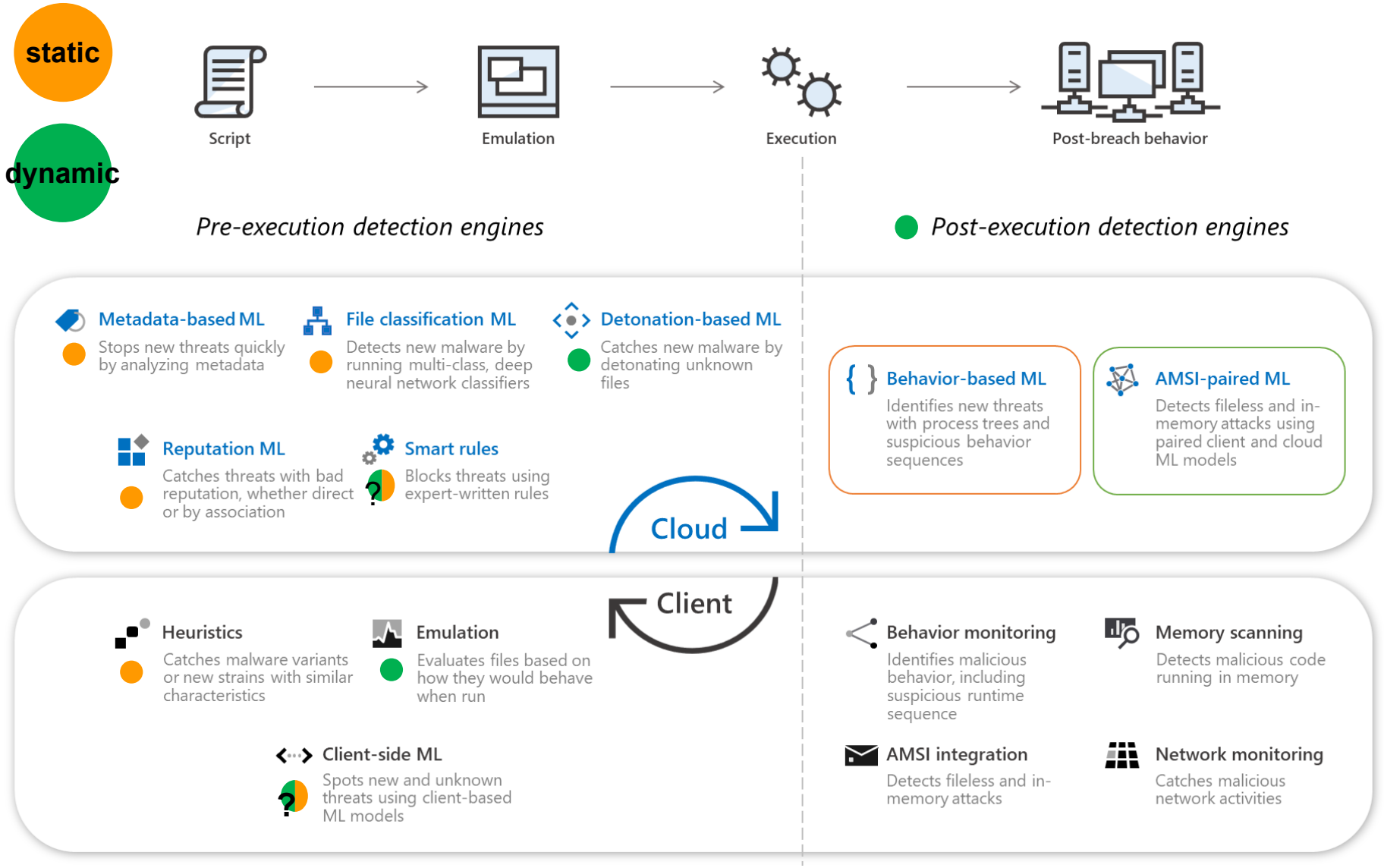
Anti-Virus Technology

- Host-based and network-based anti-virus solutions exist
- Most of them have client- (local) and cloud-based components
- Cloud-based component provides «instant-update» since signatures are hosted in the cloud
  - Usually, there is a local cache in case connectivity is lost
  - Cache often contains "most relevant" signatures only
- Bandwidth limitation – At first, only meta data is submitted
  - the file's unique identifier: (fuzzy) hash(es)/fingerprints
  - data about how the file came to be in the system
  - how the file behaved (behavioral blocker only)
- Unknown files might be temporarily quarantined on the client machine and the file sent to the cloud for analysis (automated and eventually also manual)

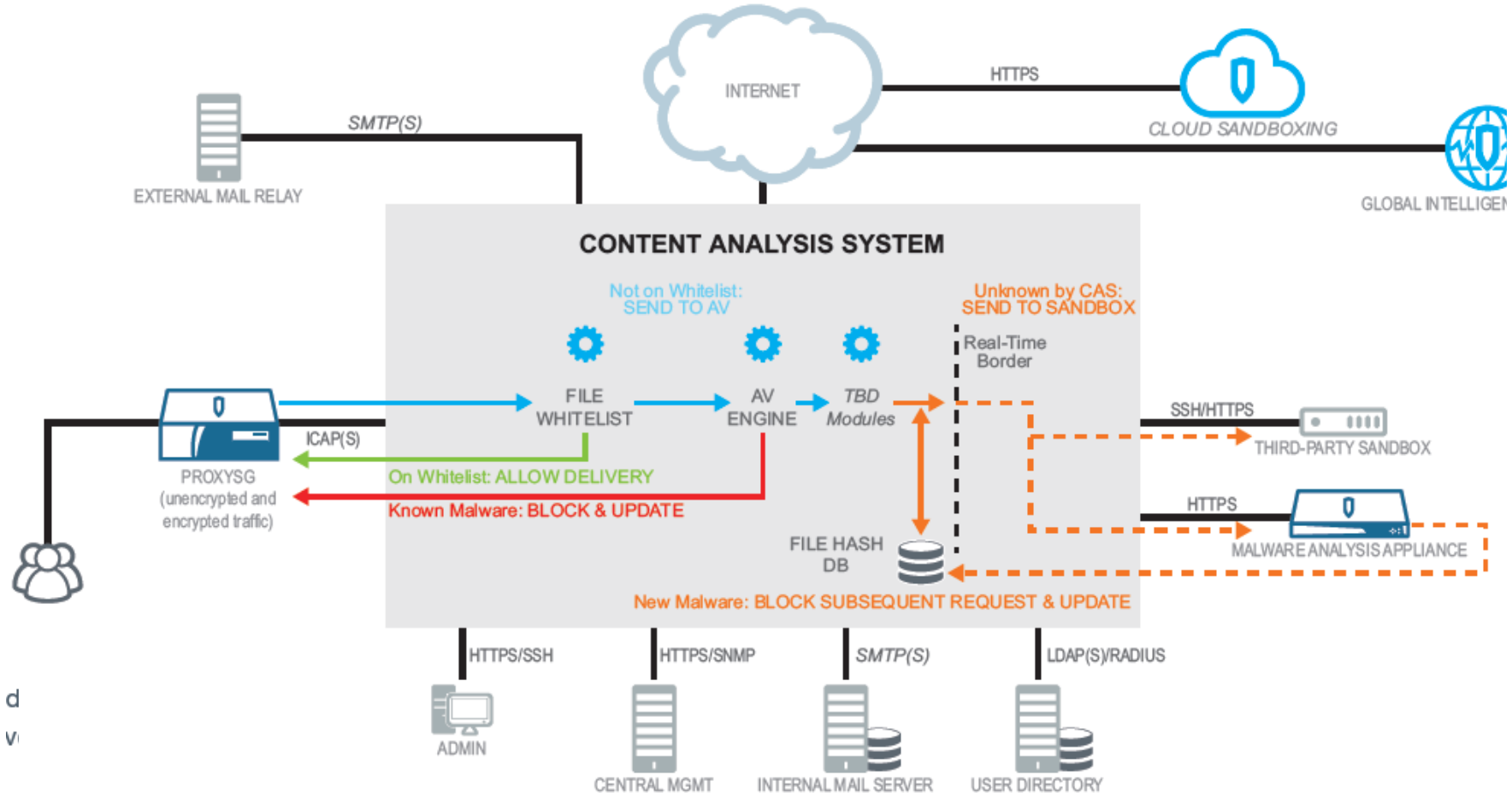
Architecture: Layered Approach



AV System Architecture – Client + Cloud



AV System Architecture – Network



Signatures

Anti-Virus Technology – Signatures (Traditional)

- Traditionally, a signature captures the unique characteristics of a malware at the byte-level
- Most common types of traditional signatures that are still in use are hashes, fuzzy-hashes, and characteristic byte sequence(s)
- Signature matching depends on the signature
  - Does the file match a signature? (byte sequences)
  - Does the signature of the sample match a hash in the database? (hash)
- Up to tens of thousands of signatures added every day
  - Clever data structures and storage formats enable anti-virus solutions to check a sample vs. all signatures in <30 ms
- Signatures often supported by whitelisting of known-good files that would trigger an alert otherwise and/or context (file size, origin,…)

Zurich University
of Applied Sciences

zh
aw

Signatures - Hashes

Signatures - Hashes

- MD5, SHA-1 or other cryptographic hash functions can be used to check whether file (or binary blob) is known to be malicious or benign
- Problem: Polymorphic/mutating malware may change its code
  - Changing one bit of the input results in a different signature
- Today, it is mainly used to prevent the analysis of samples that are already known (benign or malicious) => pre-filter / speedup
  - For cloud-based solutions, known files are not sent to the cloud
  - For solutions with sandboxes, unknown samples might be sent to the sandbox
- A signature that is more robust to modifications of the malware would be great

Zurich University
of Applied Sciences

zh
aw

Signatures – Fuzzy Hashes

Finding Similar Files – Fuzzy Hashes

- Most accurate approach to compare the byte content of two files is a side-by-side comparison looking for (byte-wise) differences
  - E.g., using a mix of xxd and diff
- Does not scale once we move from comparing two files to comparing one file to many (known malicious) files
- More efficient: Generate a fuzzy hash (=signature) for the file and compare it to the stored signatures
- Different approaches to compute fuzzy hashes exist
  - Context-triggered piecewise hashes (CTPH) (e.g., ssdeep, MRS, mrsh-v2, bbHash)
  - Other similarity digest approaches exist, for example sdhash and tlsh

Context Triggered Piecewise Hashes (CPTH)

- Basic idea:
  1. Segment a file into pieces (e.g., blocks of size n-bytes)
  2. "Hash" each piece
  3. Compile a hash from the "hashes" (e.g., concatenate them)

- What needs to be defined?
  - What alphabet to be used for the signature (optional)
    - Alphabets like base64 simplify inspection of the signatures (human readable)
  - How we want to segment the file into "summarizeable" pieces
  - Technique for summarizing a piece and form the hash composed of alphabet characters

First Attempt – Blockwise, 4-byte Blocks

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	0	1
97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	48	49
394				410				426				442				458				474				340			
K				a				q				6				K				a				U			
Kaq6KaU																											

a	b	c	d	e	f	g	h	i	h	e	l	l	o	o	p	q	r	s	t	u	v	w	x	y	z	0	1
97	98	99	100	101	102	103	104	105	104	101	108	108	111	111	112	113	114	115	116	117	118	119	120	121	122	48	49
394				410				418				442				458				474				340			
K				a				i				6				K				a				U			
Kai6KaU																											

Different block and different «summary» => GOOD!

Compare: Ka**q**6KaU with Ka**i**6KaU **=> similar!**

Different block but same «summary» => **more collision resistant** «summary function»

file content
byte values (ASCII)
sum of ASCII values
base64 char of ascii sum modulo 64
signature

First Attempt – Blockwise, 4-byte Blocks

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	0	1
97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	48	49
394				410				426				442				458				474				340			
K				a				q				6				K				a				U			
Kaq6KaU																											

a	b	c	d	e	f	g	h	i	j	k	l	m	n	h	o	p	q	r	s	t	u	v	w	x	y	z	0	1	N/A	N/A	N/A
97	98	99	100	101	102	103	104	105	106	107	108	109	110	104	111	112	113	114	115	116	117	118	119	120	121	122	48	49	0	0	0
394				410				426				434				454				470				411				49			
K				a				q				y				G				W				b				x			
KaqyGWb																															

Compare: Kaq**6KaU** with Kaq**yGWbx** **=> NOT similar!**

- There is similar content but not at the 4-byte block level
- Using fixed blocks is not the best approach…

file content
byte values (ASCII)
sum of ASCII values
base64 char of ascii sum modulo 64
signature

2nd Attempt: Context Triggered Piecewise Hashes (CPTH)

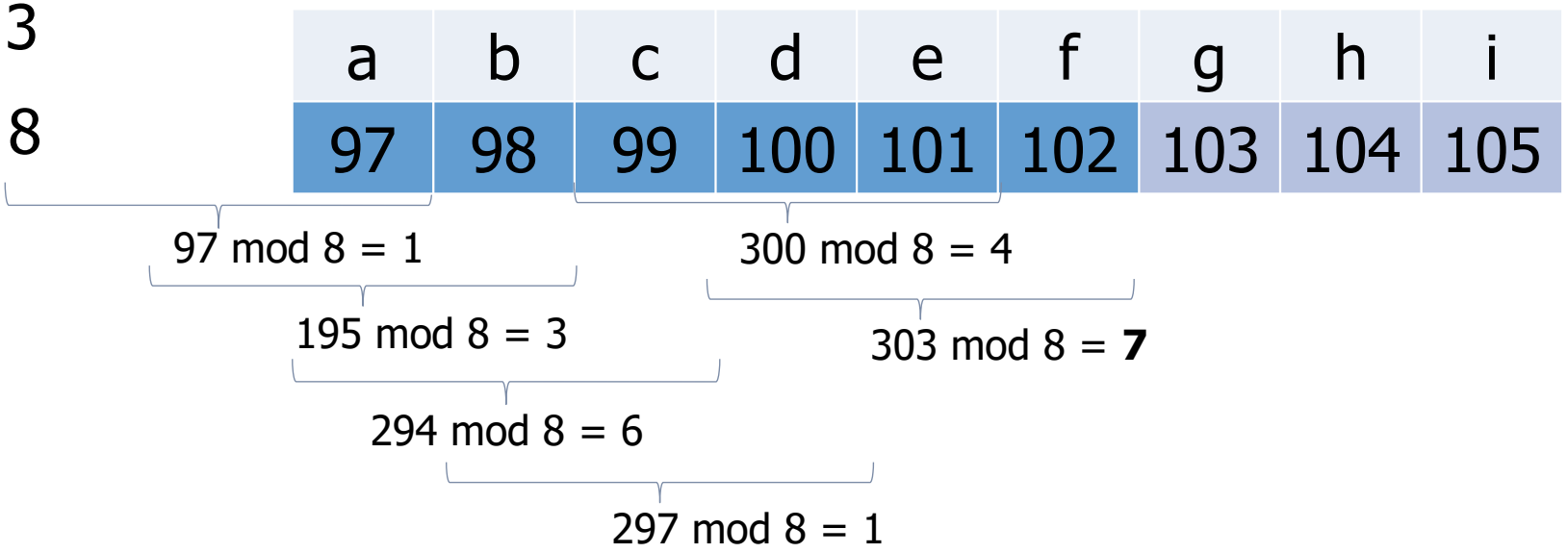
- Instead of fixed block boundaries, the content (context) determines the bounds
- One way to do this is by using a rolling hash (e.g., as in ssdeep and spamsum)

- Example of a simplistic rolling hash:

Fixed window size: 3

Calculation: ∑ mod 8

Trigger value: **7**



2nd Attempt: Context Triggered Piecewise Hashes (CPTH)

97	195	294	297	300	303	306	309	312	315	318	321	324	327	330	333	336	339	342	345	348	351	354	357	360	363	291	219
1	3	6	1	4	7	2	5	0	3	6	1	4	7	2	5	0	3	6	1	4	7	2	5	0	3	3	3
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	0	1
97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	48	49
597						852								916								579					
V						U								U								D					
VUUD																											

insert a ‘h’

97	195	294	297	300	303	306	309	312	315	318	321	324	327	323	325	327	336	339	342	345	348	351	354	357	360	363	291	219
1	3	6	1	4	7	2	5	0	3	6	1	4	7	3	5	7	0	3	6	1	4	7	2	5	0	3	3	3
a	b	c	d	e	f	g	h	i	j	k	l	m	n	h	o	p	q	r	s	t	u	v	w	x	y	z	0	1
97	98	99	100	101	102	103	104	105	106	107	108	109	110	104	111	112	113	114	115	116	117	118	119	120	121	122	48	49
597						852								327			693						579					
V						U								H			1						D					
VUH1D																												

Compare: VU**U**D with VU**H1**D **=> similar!**

- More resilient to insertions – recovers if similar content is seen after a difference
- Window size and other factors determine minimal file size to produce meaningful results

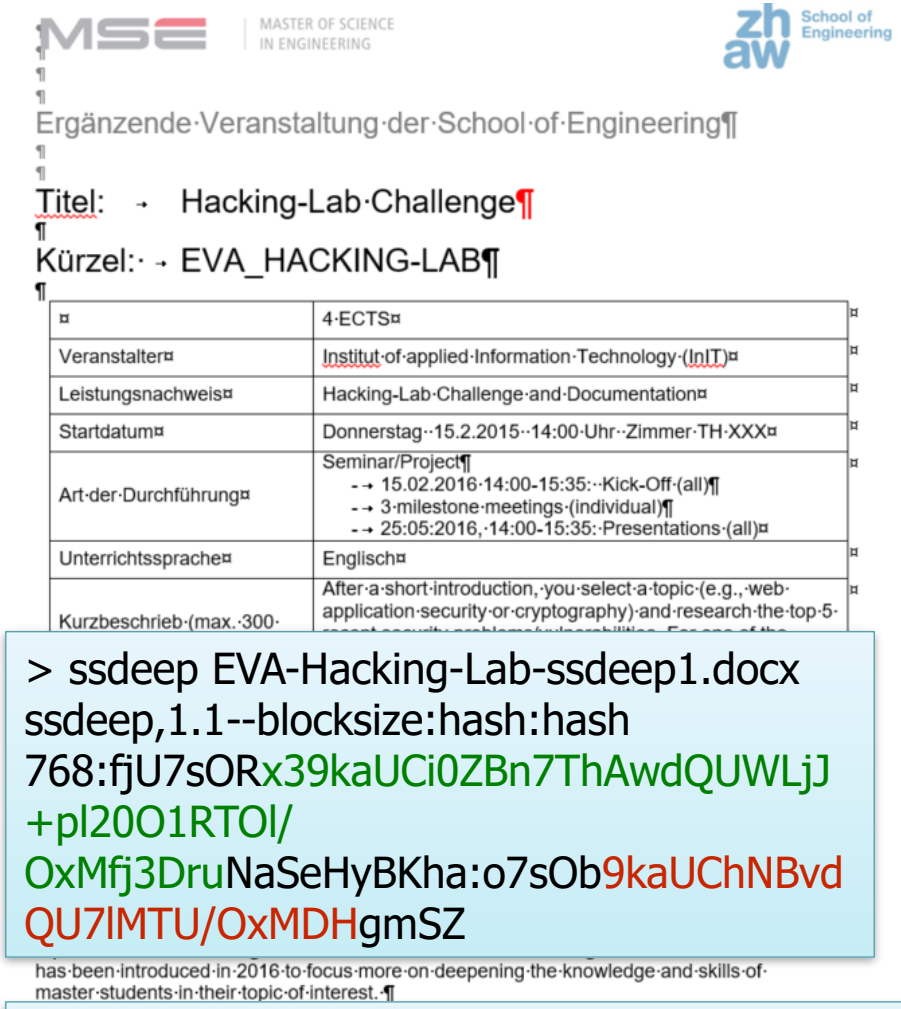
Fuzzy Hashes – CTPH Summary

- Context Triggered Piecewise Hashes (CPTH)
  - First proposed and implemented (ssdeep) by Kornblum
  - Rolling hash - r_p at position p produces a pseudo-random value based only on the current context (the last s bytes) of the input

$$r_p = F(b_p, b_{p-1}, b_{p-2}, \ldots, b_{p-s})$$

  - r_p is used to decide whether p is a reset point (block boundary)
  - The blocks are then hashed and summarized in an overall hash
- How different are the files (hashes)?
  - ssdeep uses the edit distance between the hashes as a distance measure
- Effective in finding samples where many blocks are similar and have not been moved around too much

Real-World Example With ssdeep



MSE | MASTER OF SCIENCE IN ENGINEERING — zh aw School of Engineering

Ergänzende Veranstaltung der School of Engineering

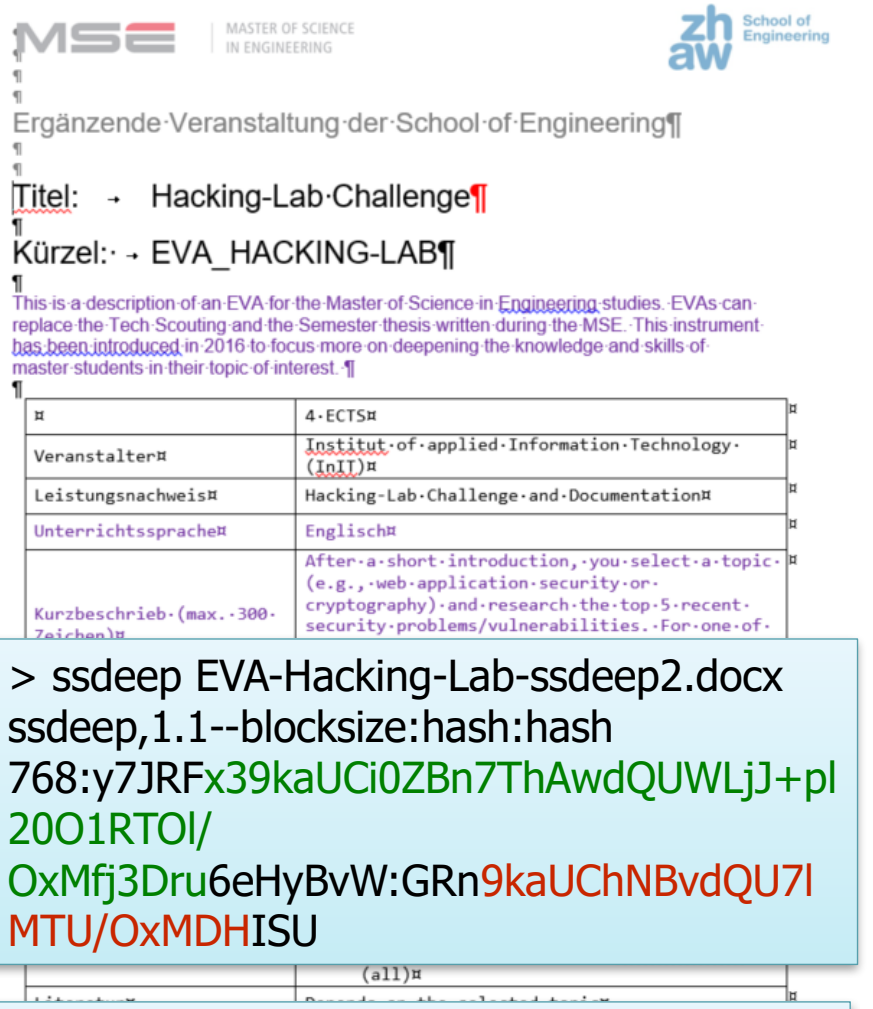
Titel: → Hacking-Lab Challenge

Kürzel: → EVA_HACKING-LAB

	4 ECTS
Veranstalter	Institut of applied Information Technology (InIT)
Leistungsnachweis	Hacking-Lab Challenge and Documentation
Startdatum	Donnerstag 15.2.2015 14:00 Uhr Zimmer TH XXX
Art der Durchführung	Seminar/Project → 15.02.2016 14:00-15:35: Kick-Off (all) → 3 milestone meetings (individual) → 25:05:2016, 14:00-15:35: Presentations (all)
Unterrichtssprache	Englisch
Kurzbeschrieb (max. 300	After a short introduction, you select a topic (e.g., web application security or cryptography) and research the top 5

has been introduced in 2016 to focus more on deepening the knowledge and skills of master students in their topic of interest.

> ssdeep EVA-Hacking-Lab-ssdeep1.docx
> ssdeep,1.1--blocksize:hash:hash
> 768:fjU7sORx39kaUCi0ZBn7ThAwdQUWLjJ+pl20O1RTOl/OxMfj3DruNaSeHyBKha:o7sOb9kaUChNBvdQU7lMTU/OxMDHgmSZ



MSE | MASTER OF SCIENCE IN ENGINEERING — zh aw School of Engineering

Ergänzende Veranstaltung der School of Engineering

Titel: → Hacking-Lab Challenge

Kürzel: → EVA_HACKING-LAB

This is a description of an EVA for the Master of Science in Engineering studies. EVAs can replace the Tech Scouting and the Semester thesis written during the MSE. This instrument has been introduced in 2016 to focus more on deepening the knowledge and skills of master students in their topic of interest.

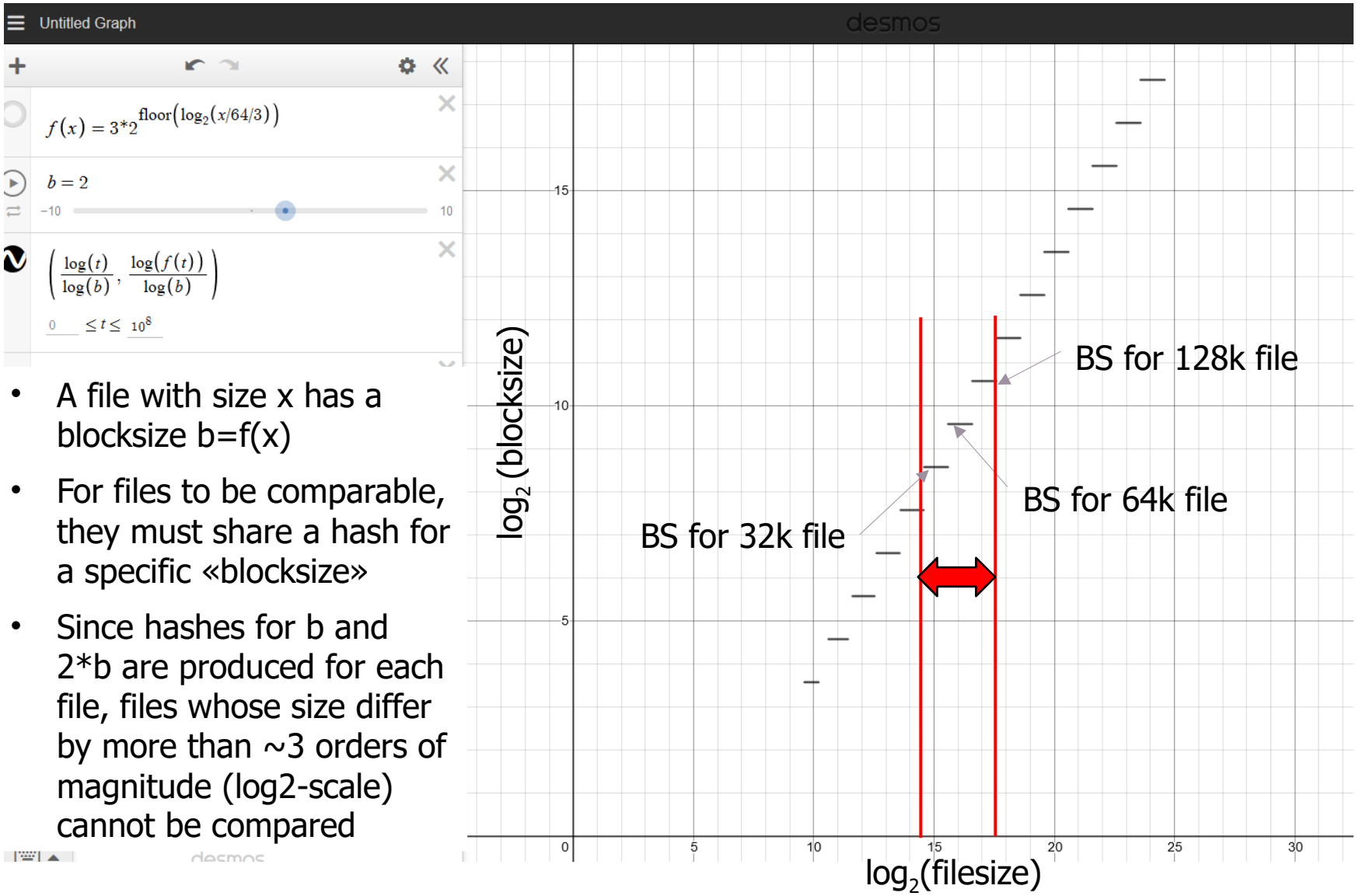
	4 ECTS
Veranstalter	Institut of applied Information Technology (InIT)
Leistungsnachweis	Hacking-Lab Challenge and Documentation
Unterrichtssprache	Englisch
Kurzbeschrieb (max. 300 Zeichen)	After a short introduction, you select a topic (e.g., web application security or cryptography) and research the top 5 recent security problems/vulnerabilities. For one of

(all)

> ssdeep EVA-Hacking-Lab-ssdeep2.docx
> ssdeep,1.1--blocksize:hash:hash
> 768:y7JRFx39kaUCi0ZBn7ThAwdQUWLjJ+pl20O1RTOl/OxMfj3Dru6eHyBvW:GRn9kaUChNBvdQU7lMTU/OxMDHISU

> ssdeep –m sig.txt EVA-Hacking-Lab-ssdeep2.docx
> EVA-Hacking-Lab-ssdeep2.docx **matches** sig.txt:EVA-Hacking-Lab-ssdeep1.docx (86)

ssdeep and Blocksizes



- A file with size x has a blocksize b=f(x)
- For files to be comparable, they must share a hash for a specific «blocksize»
- Since hashes for b and 2*b are produced for each file, files whose size differ by more than ~3 orders of magnitude (log2-scale) cannot be compared

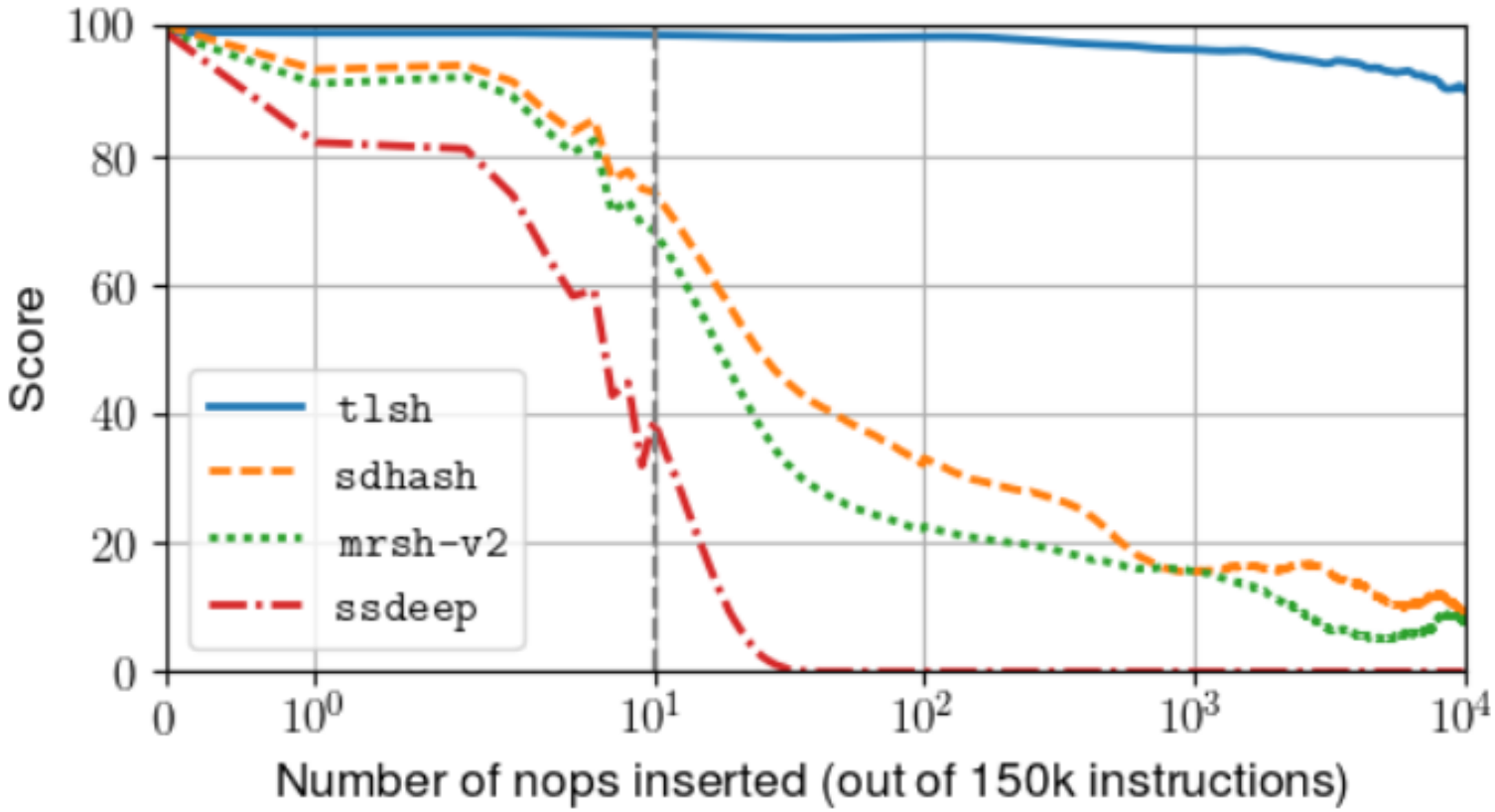
Fuzzy Hashes – Other Approaches

- Similarity digest hashing (sdhash)
  - Looks for statistically improbable sequences of 64 bytes (features)
    - A file's feature set file is represented in Bloom filters
    - Size of the hash depends on the size of the input data
  - Effective in finding samples where some parts are copied from the same source (partial matching)
- Trendmicro locality sensitive hashing (TLSH)
  - Based on the frequency distribution of n-grams (substrings of n bytes)
  - Effective in finding samples that make use of the same building blocks
    - For example, text documents written with the same language (same words)
    - For example, binaries where code blocks have been re-ordered

Fuzzy Hashes – Usage, Security and Performance

- Usage:
  - Anti-virus products make use of this but there is little to no information about it (see notes)
- Security:
  - It should be hard to make a file "not similar" anymore with minimal changes only
  - Most fuzzy hashing schemes have weaknesses (see notes)
- Performance:
  - Originally, ssdeep was considered as de-facto standard
  - Recent research showed that it might perform badly in many scenarios relevant to malware
    - Recognizing object files embedded inside a statically linked binary
    - Compilation of the malware with a different compiler (and flags)
    - Binaries with few small changes applied at the assembly level
    - Binaries compiled from different versions of the same software
    - …

Similarity Detection Performance - Example



Source: Fabio Pagani et al., 2018

Signatures – Byte Sequences

Characteristic Byte Sequence - Example

Source: https://www.kaspersky.com/blog/signature-virus-disinfection/13233/

Virus Name	*String Pattern (Signature)*
Accom.1280	89C3 B440 8A2E 2004 8A0E 2104 BA00 05CD 21E8 D500 BF50 04CD
Die.448	B440 B9E8 0133 D2CD 2172 1126 8955 15B4 40B9 0500 BA5A 01CD
Xany.979	8B96 0906 B000 E85C FF8B D5B9 D303 E864 FFC6 8602 0401 F8C3

Source: B. Rad et al., *Evolution of Computer Virus Concealment and Anti-Virus Techniques: A Short Survey*

Anti-Virus Technology – Signature Rule Languages

- Different forms of signatures based on byte-sequences exist
- One signature format is YARA
- YARA - YARA helps malware researchers to identify and classify malware samples
- Open-source tool
- Multi-platform scan engine
- Many more use cases:
  - Microsoft Office document analysis (olevba.py)
  - Forensics (yaraPCAP)
  - Intrusion detection (Hipara)

```
global private rule zip_malware_size {
    meta:
        description = "Size of all samples is lower than 1MB –
                       setting limit to 3MB"
    condition:
        uint16(0) == 0x4B50 and filesize < 3MB
}

rule apt_equation_exploitlib_mutexes {
 meta:
    copyright = “Kaspersky Lab”
    description = “Rule to detect Equation
                   group’s Exploitation library”
    version = “1.0”
    last_modified = “2015-02-16”
    reference = “https://securelist.com/blog/”
 strings:
    $mz=“MZ”
    $a1=“prkMtx” wide
    $a2=“cnFormSyncExFBC” wide
    $a3=“cnFormVoidFBC” wide
    $a4=“cnFormSyncExFBC”
    $a5=“cnFormVoidFBC”
 condition:
    (($mz at 0) and any of ($a*))
}
```

Heuristics

Heuristics - Example

```
view plain  print  ?

Rule A
An API call to RtlMoveMemory with a string of "SOFTWARE\Classes\http\shell\open\commandV"

Rule B
An API call to CreateMutexA with a string of ")!VoqA.I4"

Rule C
An API call to GetSystemDirectory

if ( Rule A then Rule B then Rule C )
then
Process  = PoisonIvy

Keribos Output
Rule A
simple.exe    | 00401447 | RtlMoveMemory(0012F458, 0040162F: "SOFTWARE\Classes\http\shell\open\commandV", 00000028) ret
.............
Rule B
simple.exe    | 0040155D | CreateMutexA(00000000, 00000000, 0012F43B: ")!VoqA.I4") returns: 0000003C
.............
Rule C
simple.exe    | 0040188F | GetSystemDirectoryA(0012F6F1, 000000FF) returns: 00000013
```

Rule A

Rule B

Rule C

IF the rules A, B and C match, then it is Poison Ivy

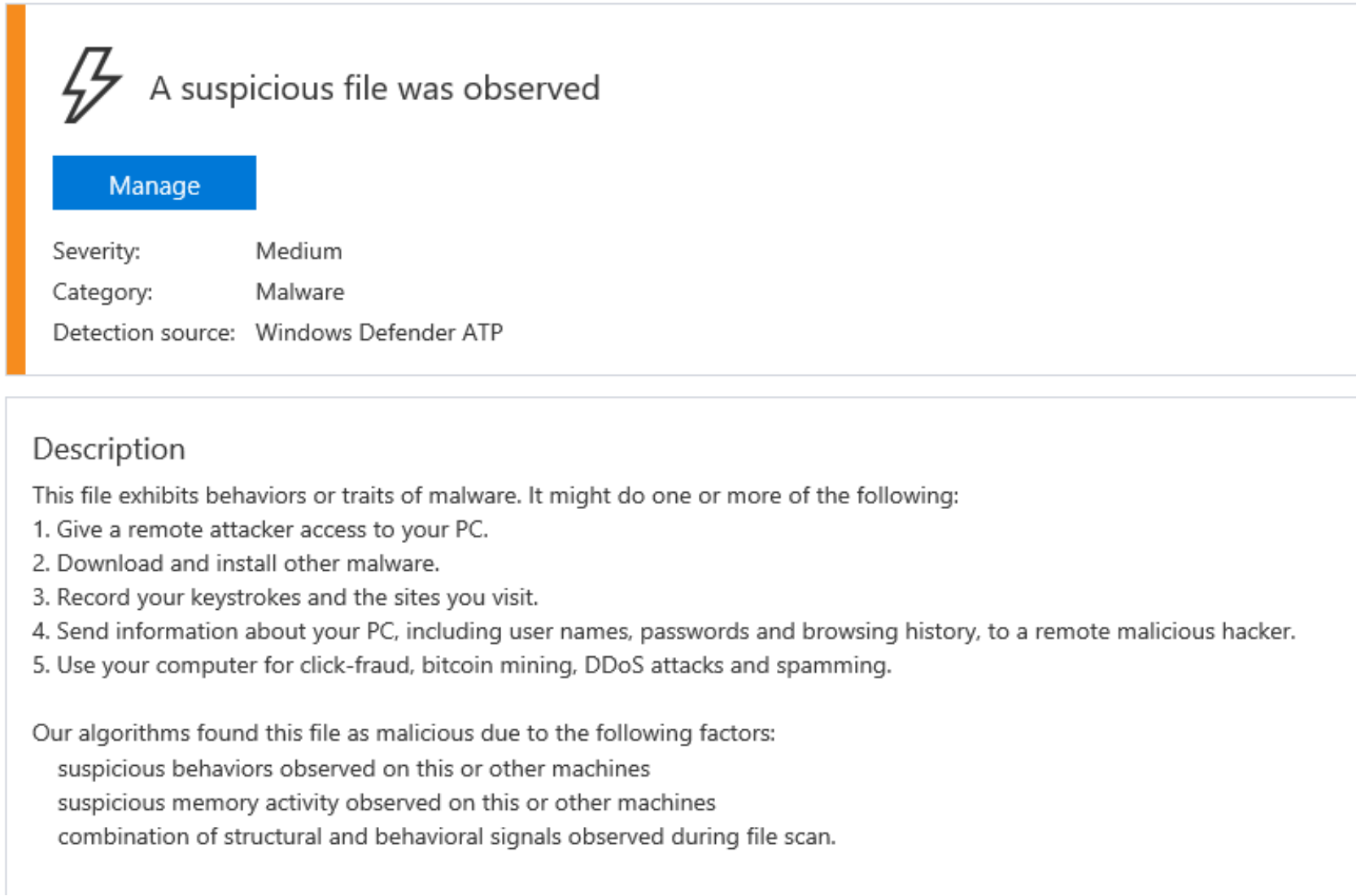
Source: http://hooked-on-mnemonics.blogspot.com/2011/01/intro-to-creating-anti-virus-signatures.html

Heuristics-based Detection

- Heuristic-based engines make use of a set of rules and weighing methods written by domain experts
- Static heuristic analysis is usually founded on the analysis of file structure (metadata) and code organization
  - E.g., suspicious section characteristics (e.g., writable code section), uncommon section names, import from KERNEL32.DLL by ordinal, …
- Dynamic heuristic analysis performs emulation of virus code to extract the features required to evaluate the rules
- Each AV engine uses different algorithms and different proprietary techniques
- Behavioral-based and heuristics-based are often used interchangeably
  - Heuristics is more general as it is not limited to behavioral features
  - Heuristic engines are usually run prior to the runtime-behavioral analysis

Behaviour

Behavioral Detection - Example



Source: https://www.microsoft.com/security/blog/2017/08/03/windows-defender-atp-machine-learning-detecting-new-and-unusual-breach-activity/

Behavioral Detection

- Behavioral (sematic) detection is unaffected by changing a malware's form – it is based on what the malware does
- Behavior on the host and on the network is sometimes considered separately by different engines
- Examples of monitored behavior:
  - HOST: Files created or modified by the malware
  - HOST: Specific changes made to the registry
  - HOST: Processes spawned
  - NET: Network sockets created
  - NET: Network connections to specific hosts, domains (or naming schemes)
  - NET: Structure of the communication protocol used
    - message length, encoding, header information etc.

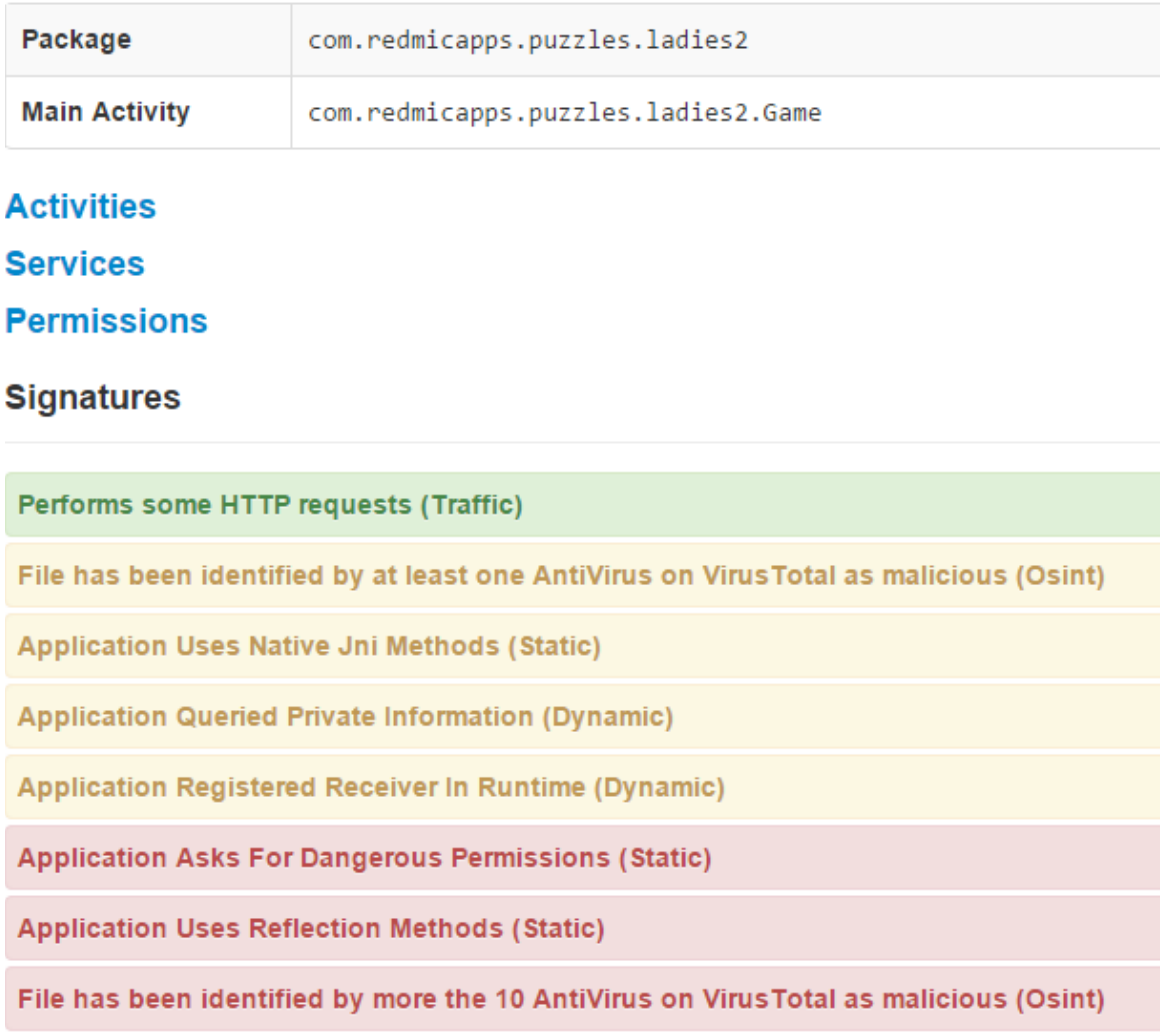
Analysis in a «Sandbox»

Behavioral Detection - Sandboxes

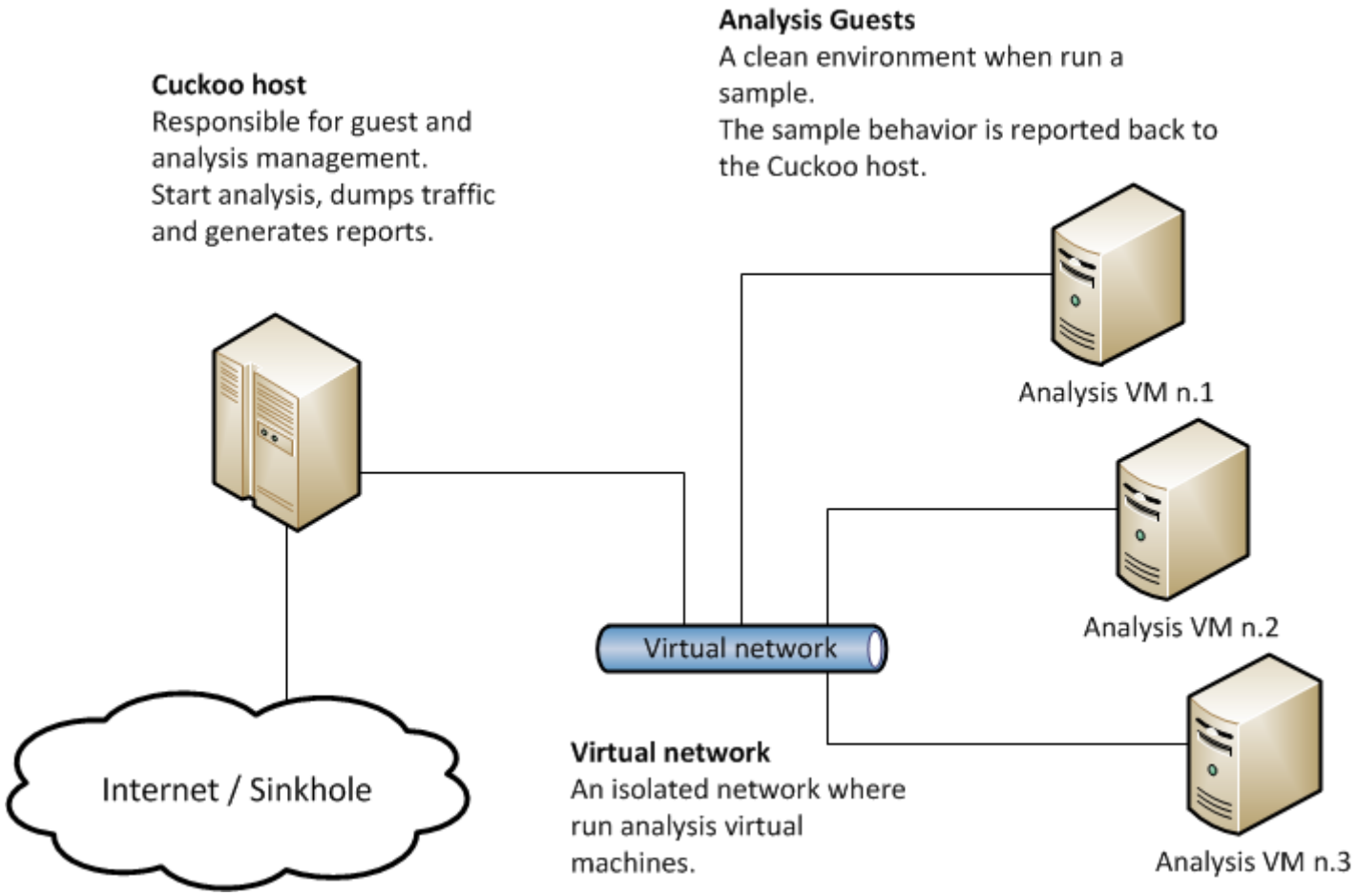
- Malware Sandbox - Captures the malicious program sample in a controlled testing environment (sandbox)
- Behavior can be studied and analyzed without affecting other systems
  - Monitor all operating system calls
    - file operations, process creation, network access, registry access etc.
  - Monitor network activity
    - Contacted end points, protocols used, traffic content, use of encryption,…
- Assumption: Malware behaves as it would in the wild

Cuckoo Sandbox

- Cuckoo Sandbox is a malware analysis system
  - Analysis of files for Windows, OS X, Linux, and Android
- By default, it can:
  - Analyze many different malicious file types and websites
  - Trace API calls and general behavior
  - Dump and analyze network traffic, even when encrypted
  - Perform advanced memory analysis with integrated support for Volatility



Cuckoo Sandbox



Analysis in a «Sandbox»

Reputation-based Analysis

- Based on what we know about a resource (e.g., file, URL, IP)
- Reputation based on things like prevalence, age, origin
  - Prevalence: A file is used on many computers around the world
    - => contributes to its reputation in a slightly positive way
  - Age: A domain was recently registered, and little is known about it at the moment
    - neutral or negative for its reputation
  - Origin: A file downloaded from the Internet is to be executed
    - negative for its reputation
  - Prevalence & Age: A file is used on many hosts and has been present for over a year now
    - contributes to its reputation in a positive way
- For malware files: Reputation systems cause a dilemma
  - Mutate more -> bad reputation/bad prevalence ->suspicious
  - Mutate less -> easy detection by signatures

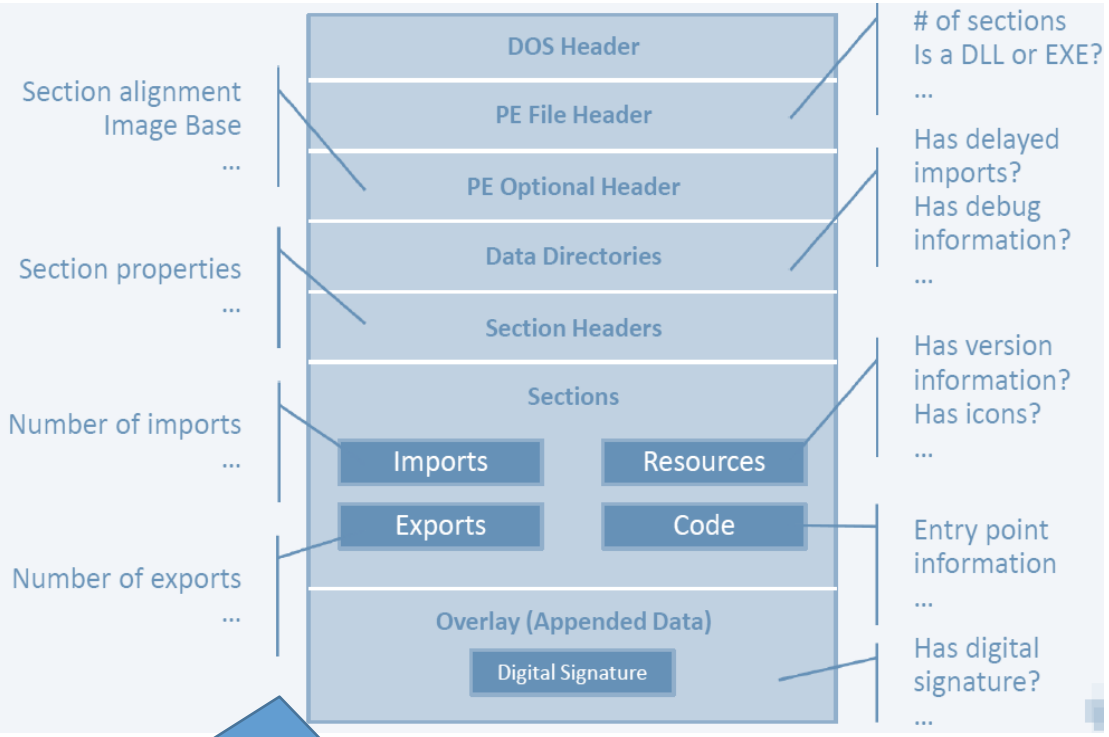
Zurich University
of Applied Sciences

zh
aw

Anti-Virus and ML

Next-Generation «Signatures» - Machine Learning

- Signatures, heuristics, or behavior are learned instead of crafted by experts
- Trains a model with millions and millions of data points
- Data points are from static (data, files) or dynamic (behavior) analysis
- Is a very active research field - some useful lessons learned:
  - https://www.microsoft.com/security/blog/2018/08/09/protecting-the-protector-hardening-machine-learning-defenses-against-adversarial-attacks/



- Static analysis examines "attributes" of the file without running the file
- Characteristic attribute combinations are learned Cylance was pioneering this

Evasion

Generic Evasion - File Formats

- Renaming of the Malware, e.g., from .exe to .hex
  - Requires social engineering to rename it back and execute it
  - Renaming is quite «normal» since email filters often block .exe or other «problematic» files => Mainly to circumvent blacklisting of file extensions
- To scan a file and to know what detection mechanisms to apply, an anti-virus system must «understand» many different file formats
  - Analysis of file formats of executables and data files (e.g., PDF, GIF)
- Vulnerabilities in those decoders has been an attack vector in the past
  - E.g., 03/2016: McAfee Enterprise antivirus could be disabled with a specifically crafted file
  - Code for less common file formats is likely to be less «mature» and tested
- Many executable file formats in Windows (e.g., *lnk, pif, wsh)*
  - lnk: Windows shortcut file simply links to a program, may contain parameters allowing for the execution of potential malicious code, e.g., by linking to cmd.exe /C <command>.

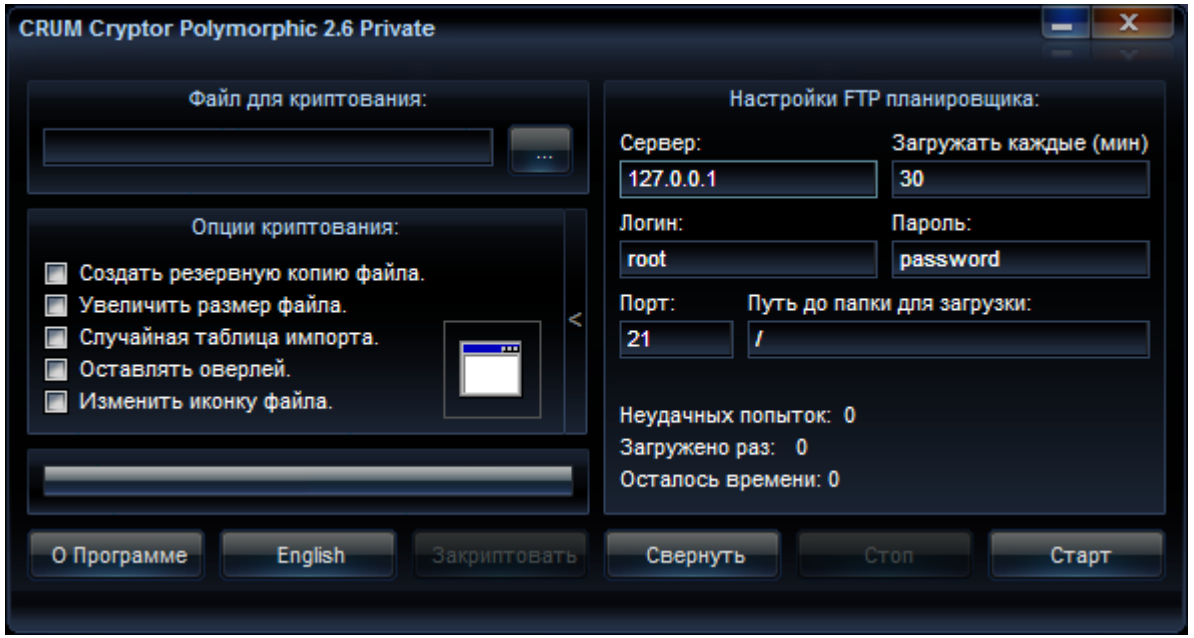
Generic Evasion - Compression

- Compressed files are «normal» and can hardly be blocked
- Compression is time consuming => AV must decompress to analyse the file
- AV gateways in the network might skip decompression and let such files either pass or block them (setting)
  - for large files
  - for nested files (ZIP Bomb: DoS potential)
  - in times of high load

  => default should be detect and block in all three cases
- Password protected (and encrypted) compressed files
  - AV can't inspect the content of the file
  - Denying such files is often not an option since this is used as an easy way to protect content sent over the Internet (e.g., email or http)

Signature Evasion - Polymorphism (1)

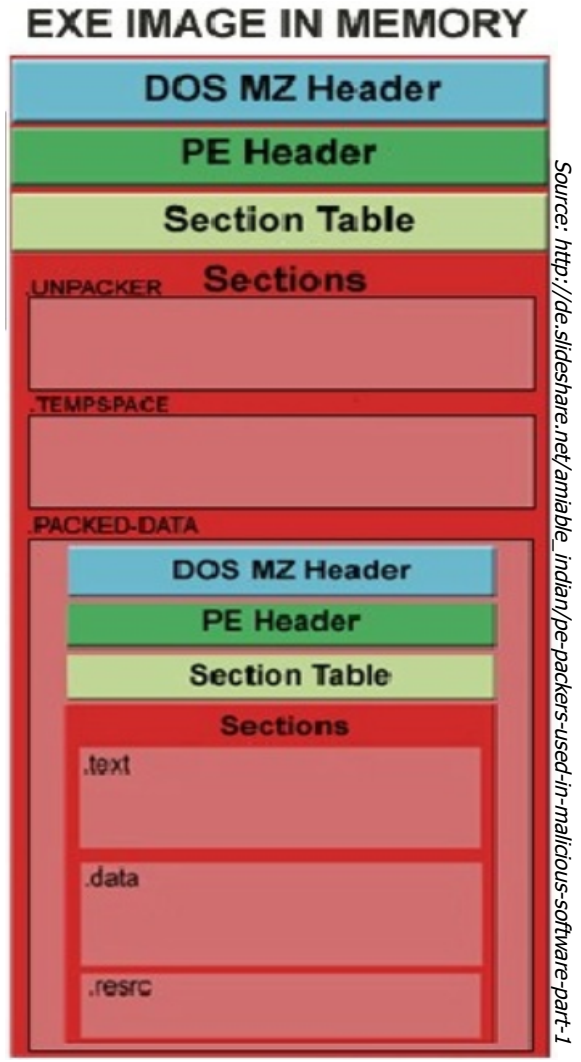
- Polymorphism - polymorphic code is code that uses a polymorphic engine to mutate while keeping the original function of the code (its semantics) the same
- For self-propagating malware, mutation engine is bundled with it, for other malware, the samples are mutated before their distribution
- Common Methods include (custom-made) cryptors and packers
- Available as tools and as-a-services



Source: https://blog.malwarebytexs.org/threat-analysis/2014/03/malware-with-packer-deception-techniques/

Signature Evasion - Polymorphism (2)

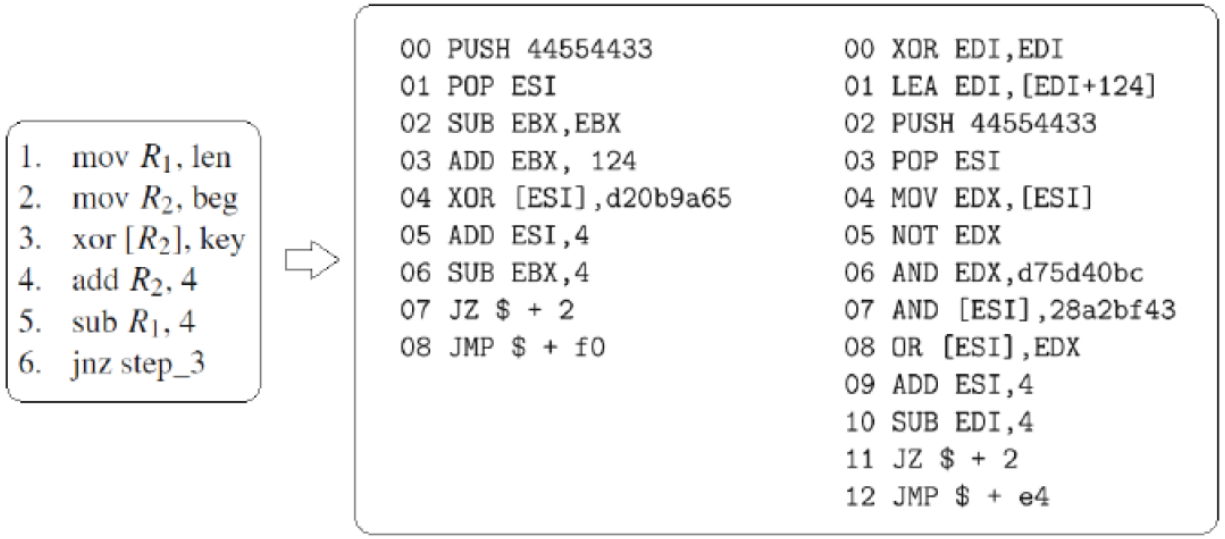
- Packers/cryptors contain a packed/encrypted part (payload) and a unpacking/decryption routine (stub) to extract+execute the payload
  - Antiviruses will attempt to generate signatures to match the stub's
  - Mutation of stub to evade signatures
    - Junk instructions that don't alter the semantics
    - Replacing instructions with others that do the same
- Example of a cryptor:
  - https://www.stringencrypt.com/



Source: http://de.slideshare.net/amiable_indian/pe-packers-used-in-malicious-software-part-1

Signature (~Heuristics/Behavior) Evasion - Metamorphism

- Metamorphism: metamorphic code is similar to polymorphic code but the whole binary including the metamorphic engine itself undergoes changes
- Idea: Translate their own binary code into an intermediate language, edit it and translate it back to machine code
- Example: A simple decryptor and two «realizations» of it:



Source: www.cs.sjsu.edu/faculty/stamp/papers/topics/topic1/teja.pdf

Evasion - Heuristics/Behavioral Detection

- The easiest way to bypass static heuristic analysis is to ensure that all the malicious code is hidden (e.g., by packing/encrypting it)
- The easiest way to bypass dynamic heuristic analysis is to hide the code in a way that the emulator is not able to unhide it (see demo)
  - If known packers/cryptors are used, it can usually unpack/decrypt the it
- For behavioral detection, the same applies as for the heuristics based detection
- In addition, for approaches monitoring the behavior at runtime (natively or in a sandbox), the following might be used to evade detection:
  - Detecting the behavioral monitoring or sandbox and behave differently
  - Outwait the behavioral monitoring by the AV solution
    - Difficult in case of endpoint security solution with always-on detection

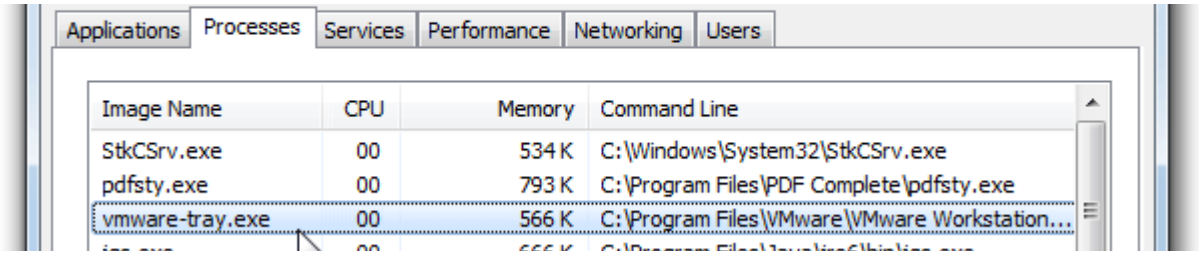
Evasion – Emulation / Sandboxes

- Fingerprint the environment like e.g., done in the browser fingerprinting project https://panopticlick.eff.org/
- Behave no-malicious if run in an analysis environment (or when specific security controls are in place)
- Virtualization-/Sandbox-specific techniques: Scan for registry entries, hardware, software (e.g., VMware Tools) or specific processes

=>Use «custom» virt. approaches or alter the footprint of existing ones

Your browser fingerprint **appears to be unique** among the 133,863 tested so far.

Currently, we estimate that your browser has a fingerprint that conveys **at least 17.03 bits of identifying information.**



Evasion - Sandboxes

- Human interaction specific – Implement some form of «CAPTCHA»
  - Wait for a certain amount of mouse clicks before continuing decryption and execution of «malicious» part (e.g., Trojan.APT.BaneChan)
  - Show dialog boxes and ask the user for a decision
  - Assess user-activities in general => is this the machine of a real user?

  => Arms race: «detecting a human» vs. «emulating a human»
- Time-specific techniques
  - Time triggers: Malicious code executes when a time condition is met (e.g., on April 1st ☺)
  - Extended sleep – Outwait the analysis sandbox

  => Sandboxes may try to modify the sleep duration, or increase the analysis period

Evasion Example – Combining Multiple Free Packers

Table 4.9: Scan results of the meterpreter reverse shell sample protected with multiple packers.

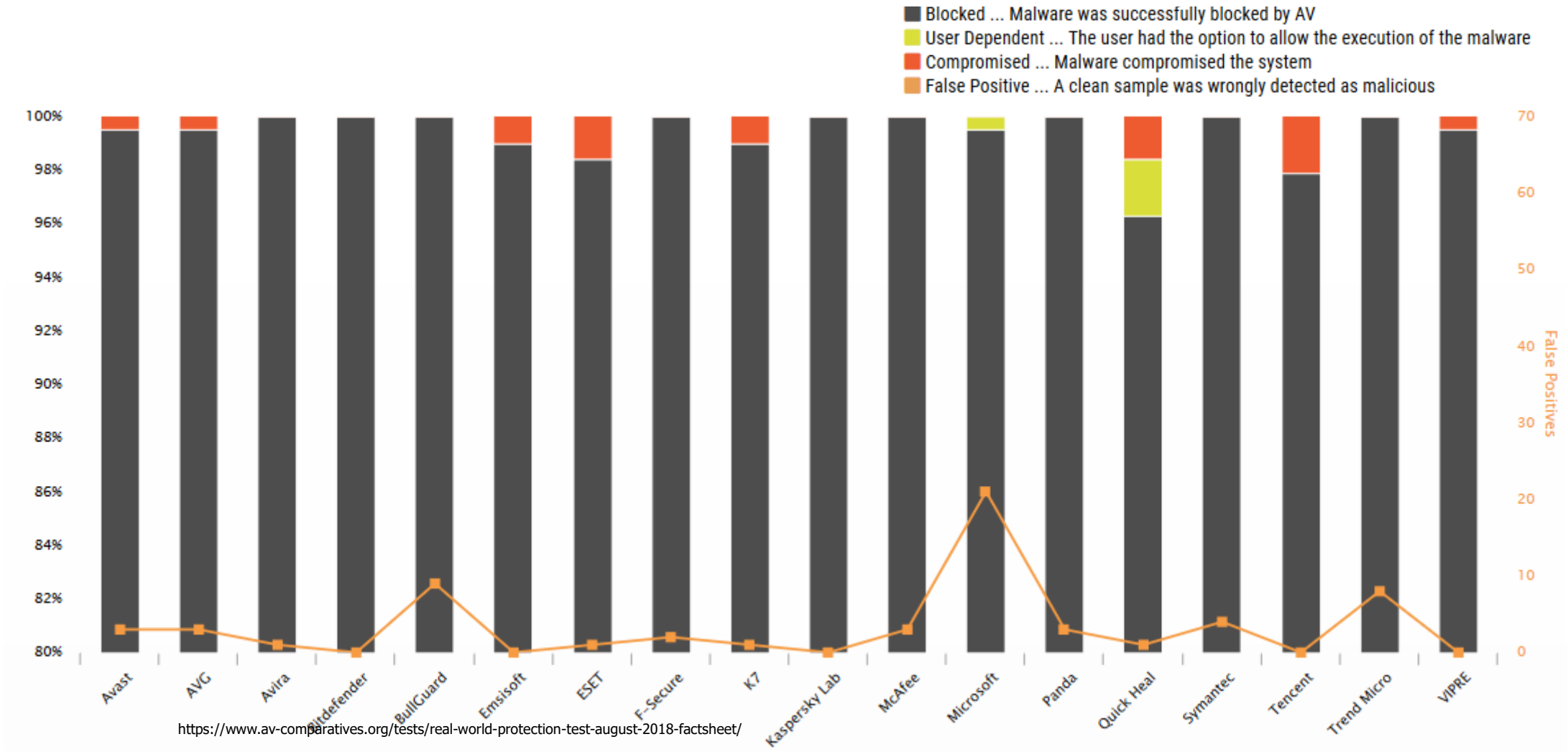
Packer	Bypasses (Suspicious)	Engines bypassed	Engines suspicious[1]
Msf Evasion	4(2)	ClamAV, EScan, FProt, McAfee	Avast, Comodo
PeCloak	6(3)	ClamAV, Comodo, FProt, Ikarus, McAfee, Sophos	Avast, EScan, FSecure
Petite	3(2)	ClamAV, EScan, FProt	Avast, FSecure,
Themida	8(3)	BitDefender, ClamAV, Comodo, EScan, FProt, Ikarus, McAfee, Sophos	Avast, Avg, FSecure
VMProtect	7(3)	ClamAV, Comodo, EScan, FProt, FSecure, Ikarus, McAfee, Sophos	Avast, BitDefender, FSecure
UPack	1(4)	Sophos	Avg, ClamAV, Comodo, Ikarus
UPX	1(2)	FProt	Avast, Avg
Obsidium	7(3)	ClamAV, Comodo, EScan, FProt, Ikarus, McAfee, Sophos	Avast, BitDefender, FSecure
PeLock	7(1)	ClamAV, Comodo, EScan, FProt, Ikarus, McAfee, Sophos	Avast
Petite	3(1)	ClamAV, EScan, FProt	Avast
Smart Packer Pro X	10(1)	Avast, Avg, BitDefender, ClamAV, Comodo, EScan, FProt, Ikarus, McAfee, Sophos	FSecure
Enigma	7(3)	BitDefender, ClamAV, Comodo, EScan, FProt, McAfee, Sophos	Avast, Avg, Ikarus

How Good is Anti-Virus?

Next Generation «Signatures» - How Good Are They?

- Difficult to say because…
  - Next generation vendors favor presenting or creating their own «tests»
  - Third party testing of next generation products are rare
    - Getting licenses for such tests seems to be difficult according to AV-Comparatives
- AV testing is a difficult business:
  - Result heavily depend on the methodology and sample set used
  - Test labs tend to follow the Anti-Malware Testing Standards Organization (AMTSO) recommendations
  - AMTSO has several documents related to testing a given protection product, from standard anti-virus to newer endpoint defense (http://www.amtso.org/)
  - Tests are done by independent labs like AV-Test, AV-Comparatives or MRG Effitas but are often sponsored by AV product vendors
    - http://www.csoonline.com/article/3167236/security/cylance-accuses-av-comparatives-and-mrg-effitas-of-fraud-and-software-piracy.html

Real-World Protection Test Results, August 2018



https://www.av-comparatives.org/tests/real-world-protection-test-august-2018-factsheet/

Anti-Virus Performance – Retrospective Test

AV comparatives	Blocked	User dependent[3]	Compromised	Proactive Protection Rate	False Alarms	Cluster
Bitdefender	1448	-	15	99%	few	1
F-Secure	1358	3	102	93%	many	1
eScan	1354	-	109	93%	many	1
Kaspersky Lab	1343	-	120	92%	Few	1
BullGuard	1259	129	75	90%	many	1
ESET	1253	-	210	86%	very few	1
Emsisoft	777	667	19	76%	many	2
Avast	985	-	478	67%	very many	2
Lavasoft	781	-	682	53%	many	3
Microsoft	772	-	691	53%	very few	3
Fortinet	742	-	721	51%	few	3
ThreatTrack	682	-	781	47%	many	-

Why are retrospective tests with modern anti-virus products usually not possible anymore?

- Heuristics and behavioral protection (offline) only
- 1463 malware samples appearing for the first time shortly after the freezing date (3rd March 2015)

Anti-Virus Performance - Summary

- No anti-virus software offers full protection from malware
- Some do «significantly» better than others but results must be considered with care - these are snapshots and depend on the test setup (malicious and benign)
- It is quite likely that you get infected by malware at some point if anti-virus is your only line of defense